ECE 2049 Lab 1: Simon Memory Game

1. Introduction

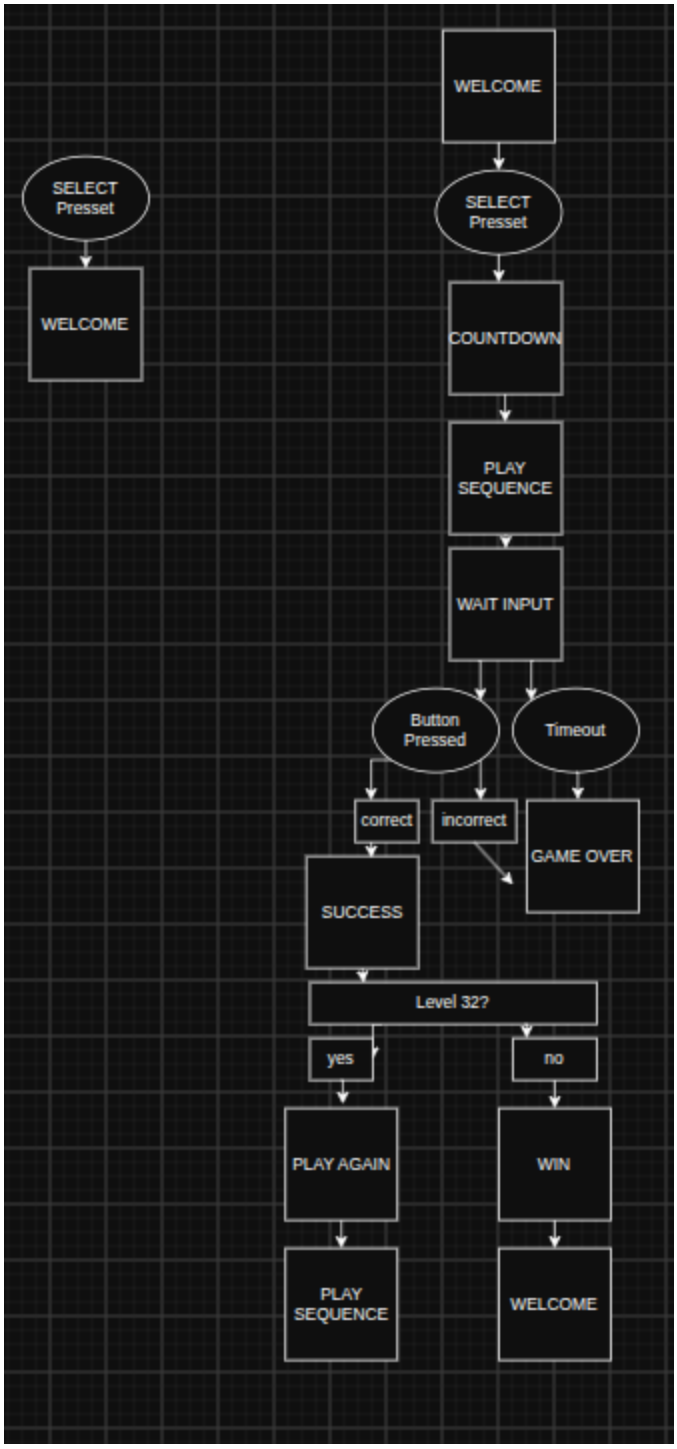
For this lab, we built an embedded version of the classic Simon memory game on an STM32 microcontroller. The hardware setup uses four LEDs, four pushbuttons, a piezo buzzer, and an LCD screen. During gameplay, the microcontroller generates a random sequence of lights and tones that the player has to memorize and repeat.

To make the game harder as it goes, the sequence grows by one step after every successful round, up to a maximum of 32 levels. If the player hits the wrong button or takes longer than five seconds to respond, the game ends. Along with the main game loop, we added a few quality-of-life features to make the interface clear: a start screen, a 3-second countdown, live button-press indicators, a history bar showing the last five inputs, and an instant reset tied to the SELECT button. Everything is combined using something called a finite-state machine (FSM) to keep the hardware peripherals from interrupting each other.

2. Operation and Program Design

After pressing SELECT on the welcome screen, a 3-second countdown runs before the LEDs and buzzer play back the first sequence, with each color mapped to a specific musical note. The player then repeats the sequence while the LCD updates the current level, active button highlights, and the last five inputs. Matching the sequence advances the level and adds a new element and a mistake or 5-second timeout triggers the game-over screen, while clearing level 32 displays a victory message.
A gameState variable drives the main loop across eight states: WELCOME, COUNTDOWN, PLAY_SEQUENCE, WAIT_INPUT, CHECK_INPUT, SUCCESS, WIN, and GAME_OVER.

Figure 1. State diagram of the Simon game. note that they are displayed without underscores for readability



WELCOME polls for the start button, COUNTDOWN handles the 3-to-1 timer, and PLAY_SEQUENCE outputs the pattern. WAIT_INPUT reads button presses and enforces the 5-second limit, while CHECK_INPUT compares inputs against the target array. SUCCESS advances the round, WIN displays the final victory screen, and GAME_OVER handles failures. In code, sequence[] stores the target pattern, playerHistory[] logs recent presses, level tracks progress, and inputIndex points to the active step. We isolated hardware logic into helper

functions like Generate_Sequence(), Read_Button(), Play_Note(), and Reset_Game() to keep the main loop clean and easy to debug/ step through.

3. Peripheral Configuration

GPIO: Four output pins drive the LEDs via LED_On(), and four active-low input pins with internal pull-ups read the pushbuttons via Button_Mask().

LCD Display: Driven by an ST7735 controller, initialized through Display_init() before enabling the backlight to show game states and input history.

Random Number Generator: HAL_RNG_GenerateRandomNumber() generates 32-bit values, which are taken modulo 4 and appended to sequence[].

Timer and Buzzer: TIM1 Channel 1 generates PWM audio (upward-counting, prescaler 0, auto-reload period 1400, initial pulse 600). Play_Note() sets each LED's tone and disables the channel afterward so notes do not overlap.

Timing: HAL_GetTick() and DWT_delayMs() manage timeouts and playback delays. Delays get shorter and shorter at higher levels to increase difficulty, and polling SELECT inside delay loops allows mid-round resets.

4. Design Evaluation and Challenges

Using an FSM was our strongest design choice, as separating states prevented timing conflicts and simplified debugging. Combining LED, audio, and LCD feedback also made sequences easier to follow at higher speeds.

Our main challenges were input tracking reliability and screen lag. Sometimes when we held a button for too long, it would register as multiple presses. We fixed this by adding Wait_Button_Release() which functions to wait for the pin to return high before accepting new inputs. Similarly, redrawing the entire LCD every loop caused noticeable stutter, so we split the output into targeted functions that only update changed indicators. With more time we would make it so that the five second timer restarted with every button input, instead of being fixed.

5. Conclusion

The final implementation successfully combines GPIO, timers, PWM, random number generation, I2C, and SPI to create an interactive Simon memory game. The finite-state machine

provides a clear structure for managing gameplay, while the LCD, LEDs, and buzzer provide feedback to the player. The design meets its main objectives.